

Fundamentos del Quadratic Assignment Problem (QAP) e integración con metaheurísticas continuas mediante *random keys*

Resumen

El *Quadratic Assignment Problem* (QAP) es un problema de optimización combinatoria en el que se **asignan n instalaciones a n localizaciones** minimizando un coste cuadrático que depende de interacciones par-a-par. Su estructura lo convierte en un benchmark recurrente en metaheurísticas y, al ser determinista, resulta adecuado para estudiar estrategias basadas en modelos *surrogate*. Este documento presenta la definición formal del QAP, su interpretación como alineamiento entre grafos ponderados, el coste computacional asociado a la evaluación, y una codificación *random keys* que permite aplicar algoritmos evolutivos continuos (AGE y DE) sin alterar sus operadores principales.

Índice

1. Definición informal	2
2. Definición formal	2
2.1. Entrada	2
2.2. Solución	2
2.3. Función objetivo	2
3. Naturaleza del problema	3
3.1. Espacio de búsqueda	3
3.2. Complejidad	3
3.3. Determinismo	3
4. Relación con grafos: alineamiento entre grafos ponderados	3
4.1. Diagrama orientativo (TikZ)	3
5. Coste computacional del QAP	3
5.1. Por qué es costoso	3
5.2. Adecuación al contexto del proyecto	4
5.3. Ventaja frente a problemas con factibilidad estructural	4
6. Implementación para AGE/DE: codificación <i>random keys</i>	4
6.1. Codificación	4
6.2. Diagrama orientativo de <i>random keys</i> (TikZ)	4
6.3. Evaluación	4
7. Pseudocódigo	5
7.1. Decodificación <i>random keys</i>	5
7.2. Evaluación QAP (minimización)	5
7.3. AGE sobre QAP	5
7.4. DE sobre QAP	6
8. Conclusión	6

1. Definición informal

Dadas n instalaciones y n localizaciones, se busca una asignación uno-a-uno que minimice el coste total derivado de las interacciones entre instalaciones. La intuición básica es:

Principio de proximidad: si dos instalaciones interactúan intensamente, conviene ubicarlas cerca.

El coste de una asignación se calcula agregando, para cada par de instalaciones, el flujo entre ellas multiplicado por la distancia entre las localizaciones asignadas.

2. Definición formal

2.1. Entrada

El QAP está determinado por:

- Una **matriz de flujos** $F \in \mathbb{R}^{n \times n}$, donde F_{ij} cuantifica la interacción (flujo) entre la instalación i y la instalación j .
- Una **matriz de distancias** $D \in \mathbb{R}^{n \times n}$, donde D_{kl} mide la distancia (o coste) entre la localización k y la localización l .

En muchas instancias prácticas D es simétrica y con diagonal nula, aunque el planteamiento general no lo requiere.

2.2. Solución

Una solución es una permutación $p \in \mathcal{S}_n$ sobre el conjunto $\{0, \dots, n-1\}$ (o $\{1, \dots, n\}$). Se interpreta como:

$p(i)$ = localización asignada a la instalación i .

La condición de permutación garantiza una asignación biyectiva: ninguna localización se repite y todas quedan ocupadas.

2.3. Función objetivo

El objetivo es minimizar el coste total:

$$\min_{p \in \mathcal{S}_n} C(p) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} F_{ij} D_{p(i)p(j)}. \quad (1)$$

Interpretación. Para cada par (i, j) , el término F_{ij} pondera la importancia de “acercar” las instalaciones; el término $D_{p(i)p(j)}$ mide la distancia real en el layout resultante.

Nota sobre doble conteo. Si F y D son simétricas, el doble sumatorio cuenta cada par dos veces. Esto no altera el conjunto de óptimos, pues introduce un factor constante.

3. Naturaleza del problema

3.1. Espacio de búsqueda

El número de asignaciones posibles es:

$$|\mathcal{S}_n| = n!.$$

El crecimiento factorial hace inviable la enumeración exhaustiva incluso para valores moderados de n .

3.2. Complejidad

El QAP es NP-hard en su forma general. En consecuencia, es habitual recurrir a metaheurísticas para encontrar soluciones de alta calidad en tiempos razonables.

3.3. Determinismo

Dadas F y D , la evaluación $C(p)$ es determinista: la misma permutación produce siempre el mismo coste. Esta propiedad es especialmente conveniente para técnicas surrogate, cuyo objetivo es aproximar una función objetivo fija.

4. Relación con grafos: alineamiento entre grafos ponderados

El QAP puede verse como el alineamiento entre dos grafos ponderados:

- Grafo de flujo G_F : nodos = instalaciones, peso en arista $(i, j) = F_{ij}$.
- Grafo de distancia G_D : nodos = localizaciones, peso en arista $(k, l) = D_{kl}$.

La permutación p define una biyección entre nodos de G_F y G_D . El coste en (1) mide el acoplamiento entre “interacciones” y “distancias” inducido por esa biyección.

4.1. Diagrama orientativo (TikZ)

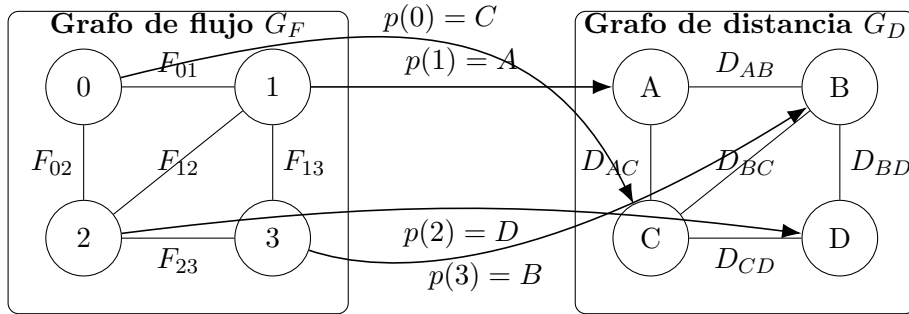


Figura 1: Interpretación del QAP como alineamiento entre un grafo de flujos (G_F) y un grafo de distancias (G_D).

5. Coste computacional del QAP

5.1. Por qué es costoso

El coste computacional proviene de dos factores complementarios:

1. **Espacio de búsqueda enorme:** hay $n!$ soluciones posibles.
2. **Evaluación cuadrática:** el cálculo directo de (1) requiere n^2 términos, por tanto cada evaluación cuesta $\mathcal{O}(n^2)$.

En metaheurísticas se realizan miles o cientos de miles de evaluaciones. Aunque $\mathcal{O}(n^2)$ es polinómico, el coste acumulado puede dominar el tiempo total (especialmente si se evalúa en bucles en Python sin vectorización).

5.2. Adecuación al contexto del proyecto

El QAP cumple los requisitos habituales para evaluar estrategias surrogate:

- **Determinista:** misma solución $\Rightarrow$ mismo coste.
- **Combinatorio y NP-hard:** espacio de búsqueda factorial.
- **Fitness no trivial:** evaluación cuadrática, acumulada en muchas evaluaciones.
- **Benchmark consolidado:** instancias estándar (p. ej., QAPLIB).

5.3. Ventaja frente a problemas con factibilidad estructural

A diferencia de problemas como Steiner Tree (con conectividad y reparación estructural), en el QAP la factibilidad se reduce a ser permutación. Esto disminuye la complejidad de implementación y el riesgo técnico, manteniendo dificultad real para metaheurísticas.

6. Implementación para AGE/DE: codificación *random keys*

6.1. Codificación

Para reutilizar algoritmos continuos (AGE y DE) sin rediseñar operadores de permutación, se representa cada solución como:

$$x \in \mathbb{R}^n,$$

y se decodifica a permutación ordenando los índices:

$$p = \text{argsort}(x). \tag{2}$$

Intuición. El orden relativo de los componentes de x define el orden de la permutación. Pequeñas variaciones en x pueden cambiar el ranking de algunos componentes y, por tanto, inducir cambios locales en la permutación.

6.2. Diagrama orientativo de *random keys* (TikZ)

6.3. Evaluación

Dado x :

1. Decodificar $x \mapsto p$ con (2).
2. Calcular $C(p)$ con (1).
3. Devolver $C(p)$ como fitness (minimización).

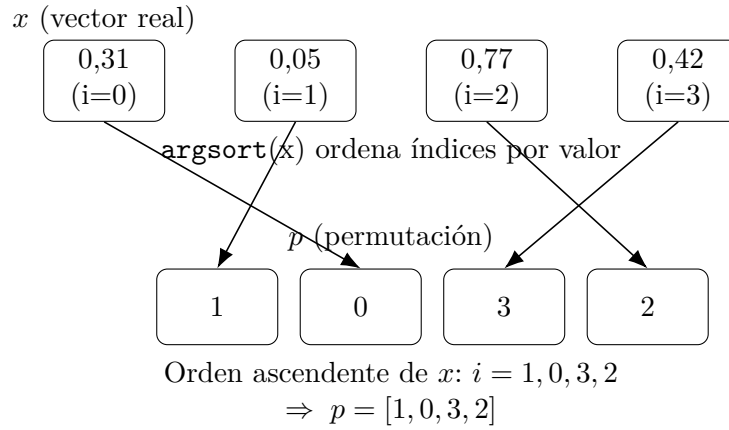


Figura 2: Decodificación *random keys*: la permutación se obtiene ordenando los índices del vector real x .

En implementación práctica (p. ej., NumPy), suele reordenarse la matriz D :

$$D_p = D[p, :][:, p],$$

y calcular el producto interno matricial:

$$C(p) = \sum_{i,j} F_{ij}(D_p)_{ij}.$$

7. Pseudocódigo

7.1. Decodificación *random keys*

```
procedure DECODE_RANDOM_KEYS(x)
  # x: vector real de tamaño n
  p <- argsort(x)
  return p
end procedure
```

7.2. Evaluación QAP (minimización)

```
procedure EVALUAR_QAP(x, F, D)
  p <- DECODE_RANDOM_KEYS(x)
  coste <- 0

  for i in 0..n-1:
    for j in 0..n-1:
      coste <- coste + F[i,j] * D[p[i], p[j]]

  return coste
end procedure
```

7.3. AGE sobre QAP

```
procedure AGE_QAP
  P <- población inicial (random keys)
  evaluar P
```

```

while evals < max_evals:
    p1, p2 <- selección (p.ej. torneo)
    h1, h2 <- cruce(p1, p2)
    h1, h2 <- mutación(h1, h2)
    evaluar h1, h2 con EVALUAR_QAP
    reemplazo estacionario según menor fitness

return mejor individuo
end procedure

```

7.4. DE sobre QAP

```

procedure DE_QAP
    P <- población inicial (random keys)
    evaluar P

    while evals < max_evals:
        for cada individuo i:
            v <- mutación diferencial
            u <- cruce entre v e i
            u <- recorte a [0,1]
            if EVALUAR_QAP(u) <= EVALUAR_QAP(i):
                reemplazar i por u

    return mejor individuo
end procedure

```

8. Conclusión

El QAP proporciona un equilibrio adecuado entre dificultad teórica (NP-hard), determinismo, coste acumulado por evaluación y simplicidad de implementación. La codificación *random keys* permite reutilizar AGE y DE (operadores continuos) delegando el componente combinatorio en una decodificación determinista mediante **argsort**. Esto lo convierte en un problema apropiado para experimentar con modelos surrogate orientados a aproximación de fitness y filtrado de candidatos.